LAB 2

Install and Run Kafka in KRaft Mode

Apache Kafka without ZooKeeper — KRaft-based metadata management

Rajesh Pasham

Objectives & Lab Topology

OBJECTIVES

1. Install Apache Kafka on a Linux machine
2. Configure a single-node KRaft cluster (combined broker and controller)
3. Start Kafka in KRaft mode and verify basic messaging

LAB TOPOLOGY

- One Linux VM or server
- One Kafka process as both Broker & Controller
- PLAINTEXT client listener → localhost:9092
- Controller listener → localhost:9093

Setup: Install Kafka & Configure KRaft Properties

Step 1: Reuse or Install Kafka

If Lab 1 was completed on the same machine, reuse the KAFKA_HOME directory. Otherwise, repeat the Java installation and Kafka download steps. The config/kraft directory contains KRaft-ready configuration files including server.properties.

Step 2: Configure KRaft Server Properties

Edit config/kraft/server.properties with these key properties:

```
process.roles=broker,controller
node.id=1
controller.quorum.voters=1@localhost:9093
listeners=PLAINTEXT://:9092,CONTROLLER://:9093
log.dirs=/tmp/kraft-combined-logs
```

Generate Cluster ID & Format KRaft Storage

Step 3: Generate a Cluster ID

From KAFKA_HOME, run the following to store a UUID in KAFKA_CLUSTER_ID:

```
KAFKA_CLUSTER_ID="$(bin/kafka-storage.sh random-uuid)"
```

Kafka documentation prescribes kafka-storage.sh random-uuid to generate a cluster ID before formatting KRaft storage.

Step 4: Format KRaft Storage Directories

Format the log directory using the generated Cluster ID:

```
bin/kafka-storage.sh format --standalone -t $KAFKA_CLUSTER_ID -c config/kraft/server.properties
```

The format step writes meta.properties with the cluster ID — Kafka will not start in KRaft mode if this step is skipped.

Start, Test & Verify the KRaft Cluster

5 Start the KRaft Broker/Controller

```
bin/kafka-server-start.sh config/kraft/server.properties
```

Confirm logs show node running as both broker & controller on port 9092.

6 Create and Inspect a Topic

```
bin/kafka-topics.sh --create --topic kraft-test --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1
```

Topic commands are identical to ZooKeeper mode.

7 Produce and Consume Messages

```
bin/kafka-console-consumer.sh --topic kraft-test --from-beginning --bootstrap-server localhost:9092
```

This loopback test validates end-to-end flow in KRaft mode.

Stop, Reset & Extend: Multi-Node Setup

Step 8: Stop KRaft Node and Reset

- Stop producer and consumer with Ctrl+C
- Stop the KRaft node with Ctrl+C in the server terminal
- Delete log directory: rm -rf /tmp/kraft-combined-logs
- Rerun kafka-storage.sh format before restarting

Deleting the log directory removes meta.properties — Kafka must be reformatted before restarting in KRaft mode.

Extension: Multi-Controller, Multi-Broker KRaft Cluster

3 dedicated controllers · 3 dedicated brokers · All on one Linux VM · Replicated topics with RF=3

Steps 9 – 15 ▶

Multi-Node: Config Files & Controller Setup

Step 9: Prepare Separate Config Files

Copy server.properties to create 6 files: controller-1/2/3.properties and broker-1/2/3.properties. Each file will get unique roles, node IDs, ports, and log directories.

Step 10: Configure Three Controller-Only Nodes

Set the following properties in each controller file (unique node.id, port, and log.dirs per controller):

```
process.roles=controller
node.id=1  (unique per controller: 1, 2, 3)
listeners=CONTROLLER://:19093  (ports: 19093, 29093, 39093)
controller.quorum.voters=1@localhost:19093,2@localhost:29093,3@localhost:39093
log.dirs=/tmp/kraft-controller-1-logs
```

Multi-Node: Broker Config & Storage Format

Step 11: Configure Three Broker-Only Nodes

Each broker uses unique node.id (4, 5, 6) — NOT in controller.quorum.voters — with a unique client port and log directory:

```
process.roles=broker
node.id=4  (brokers use 4, 5, 6)
listeners=PLAINTEXT://:9092  (ports: 9092, 9094, 9096)
controller.quorum.voters=1@localhost:19093,2@localhost:29093,3@localhost:39093
```

Step 12: Generate Cluster ID & Format All Nodes

```
CLUSTER_ID="$(bin/kafka-storage.sh random-uuid)"
bin/kafka-storage.sh format --cluster-id "$CLUSTER_ID" -c config/kraft/controller-1.properties
# Repeat for controller-2, controller-3, broker-1, broker-2, broker-3
```

Start Cluster & Create Replicated Topic

Step 13: Start Controllers, Then Brokers

Controllers (Terminals 1-3)

```
bin/kafka-server-start.sh \
  config/kraft/controller-1.properties
# Repeat for controller-2, controller-3
```

Brokers (Terminals 4-6)

```
bin/kafka-server-start.sh \
  config/kraft/broker-1.properties
# Repeat for broker-2, broker-3
```

Step 14: Create Replicated Topic & Verify

```
bin/kafka-topics.sh --create --topic kraft-multinode-test \
  --bootstrap-server localhost:9092 --partitions 3 --replication-factor 3
```

Describe output should show ReplicationFactor: 3 with ISR replicas on broker IDs 4, 5, and 6.

Step 15: Cleanup & Lab Summary

Cleanup Commands

```
# Stop all producers, consumers, brokers, controllers (Ctrl+C)
rm -rf /tmp/kraft-controller-1-logs \
       /tmp/kraft-controller-2-logs \
       /tmp/kraft-controller-3-logs \
       /tmp/kraft-broker-1-logs \
       /tmp/kraft-broker-2-logs \
       /tmp/kraft-broker-3-logs
```

Single-Node KRaft

Steps 1–8: Install, configure, start, and verify a single combined broker+controller node.

Multi-Node KRaft

Steps 9–15: Deploy 3 controllers + 3 brokers on one VM with replicated topics.

Key Takeaway

KRaft eliminates ZooKeeper dependency — metadata is managed by Kafka's own Raft quorum.